CHICAGO POLICE DEPARTMENT · CRIME DATA ANALYTICS

Use Case 4

Database Reporting and Pandas Integration

Objective · Summary tables, SQL reporting queries, stored views and charts from SQL.

Prepared for	Chicago Police Department
Prepared by	Accenture — Python Full Stack Program
Data source	chicago_crime_dataset.csv + 4 reference files
Toolchain	Python · Pandas · NumPy · Matplotlib · Seaborn
Warehouse	SQLite3 @ chicago_crime.db
Generated	17 August 2026, 11:10

1. Connecting to the warehouse

Everything in this use case is executed against the database, not against a DataFrame. The connection is opened with a raw Python DB-API connector — pymysql for MySQL, sqlite3 for the fallback backend — because the brief asks for the summary tables to be created and loaded "through the Python connector". Bulk DataFrame reads use SQLAlchemy on top of the same connection string.

This run targets **SQLite3 @ chicago_crime.db**. A MySQL server was not reachable with credentials in this environment, so the SQLite3 fallback the brief explicitly allows is in use. Nothing else changes: the DDL in sql/schema_mysql.sql and sql/schema_sqlite.sql is structurally identical, and the view definitions in sql/views.sql are portable SQL shared by both backends.

CODE

```
import numpy as np
import pandas as pd

pd.set_option('display.width', 110)

# Raw DB-API connection: pymysql.connect(...) or sqlite3.connect(...)
# depending on DB_BACKEND. See db/connection.py.
conn = dbc.get_connector()
cur = conn.cursor()

print(f"Backend : {dbc.backend_label()}")

table_sql = ("SHOW TABLES" if config.DB_BACKEND == 'mysql' else
             "SELECT name FROM sqlite_master WHERE type='table' ORDER BY name")
cur.execute(table_sql)
print("Tables  :", ", ".join(row[0] for row in cur.fetchall()))
```

OUTPUT

```
Backend : SQLite3 @ chicago_crime.db
Tables  : chicago_crime, city_community, district_ps_info, iucr, police_beat_info, summary_crime_by_category,
  summary_crime_by_community, summary_crime_yearly, ward_office
```

CODE

```
cur.execute("SELECT COUNT(*) FROM chicago_crime")
crimes = cur.fetchone()[0]

for table in ['iucr', 'city_community', 'district_ps_info',
              'police_beat_info', 'ward_office']:
    cur.execute(f"SELECT COUNT(*) FROM {table}")
    print(f"{table:20s}: {cur.fetchone()[0]:>5,} rows")
print(f"{'chicago_crime':20s}: {crimes:>5,} rows  (fact table)")
```

OUTPUT

```
iucr                :    24 rows
city_community      :    77 rows
district_ps_info    :    22 rows
police_beat_info    :   198 rows
ward_office         :    24 rows
chicago_crime       : 2,000 rows  (fact table)
```

2. Designing and populating the summary tables

What is being asked: Create summary tables in the database and load them, doing the work through the Python database connector rather than by hand.

Three pre-aggregated tables serve the reporting layer. They exist so a dashboard never has to scan the fact table for a headline number: the yearly and category rollups answer the questions below in a single indexed row read each.

SCHEMA — summary tables

```
-- sql/schema_mysql.sql (summary layer)
CREATE TABLE summary_crime_yearly (
    year         SMALLINT NOT NULL,
    total_crimes INT      NOT NULL,
    arrests      INT      NOT NULL,
    arrest_rate  DECIMAL(6,2),
    PRIMARY KEY (year)
) ENGINE=InnoDB;

CREATE TABLE summary_crime_by_category (
    primary_type VARCHAR(64) NOT NULL,
    total_crimes INT         NOT NULL,
    pct_of_total DECIMAL(6,2),
    arrests      INT,
    arrest_rate  DECIMAL(6,2),
    PRIMARY KEY (primary_type)
) ENGINE=InnoDB;

CREATE TABLE summary_crime_by_community (
    community_code INT NOT NULL,
    community_name VARCHAR(96),
    total_crimes   INT NOT NULL,
    population     INT,
    crimes_per_10k DECIMAL(12,2),
    PRIMARY KEY (community_code)
) ENGINE=InnoDB;
```

CODE

```
# Populate summary_crime_yearly straight from the fact table, using the
# connector's parameter binding rather than string-formatted SQL.
placeholder = '%s' if config.DB_BACKEND == 'mysql' else '?'

cur.execute("DELETE FROM summary_crime_yearly")
cur.execute('''
    SELECT year,
           COUNT(*)                                AS total_crimes,
           SUM(arrest)                             AS arrests,
           ROUND(SUM(arrest) * 100.0 / COUNT(*), 2) AS arrest_rate
    FROM   chicago_crime
    GROUP  BY year
    ORDER  BY year
''')
rows = cur.fetchall()

cur.executemany(
    f"INSERT INTO summary_crime_yearly "
    f"(year, total_crimes, arrests, arrest_rate) "
    f"VALUES ({placeholder}, {placeholder}, {placeholder}, {placeholder})",
    rows)
conn.commit()

print(f"{cur.rowcount if cur.rowcount > 0 else len(rows)} rows written "
      f"to summary_crime_yearly")
cur.execute("SELECT * FROM summary_crime_yearly ORDER BY year")
for row in cur.fetchall():
    print(f"  {row[0]}  crimes={row[1]:>4}  arrests={row[2]:>3}  rate={row[3]:>6}%")
```

OUTPUT

```
9 rows written to summary_crime_yearly
  2015  crimes= 227  arrests= 63  rate= 27.75%
  2016  crimes= 242  arrests= 62  rate= 25.62%
  2017  crimes= 192  arrests= 56  rate= 29.17%
  2018  crimes= 213  arrests= 72  rate=  33.8%
  2019  crimes= 236  arrests= 75  rate= 31.78%
  2020  crimes= 224  arrests= 68  rate= 30.36%
  2021  crimes= 229  arrests= 68  rate= 29.69%
  2022  crimes= 221  arrests= 72  rate= 32.58%
  2023  crimes= 216  arrests= 60  rate= 27.78%
```

CODE

```
# The other two summary tables are rebuilt the same way by the pipeline.
from db.etl import build_summary_tables

written = build_summary_tables()
for table, n in written.items():
    print(f"{table:28s}: {n:>3} rows")

print()
print(dbc.read_sql('''
    SELECT primary_type, total_crimes, pct_of_total, arrest_rate
    FROM   summary_crime_by_category
    ORDER  BY total_crimes DESC LIMIT 8
''').to_string(index=False))
```

OUTPUT

```
summary_crime_yearly        :   9 rows
summary_crime_by_category   :  16 rows
summary_crime_by_community  :  77 rows

       primary_type  total_crimes  pct_of_total  arrest_rate
              THEFT           416         20.80        14.90
            BATTERY           328         16.40        41.77
            ASSAULT           195          9.75        29.74
          NARCOTICS           178          8.90        56.74
           BURGLARY           177          8.85        12.99
            ROBBERY           165          8.25        38.79
MOTOR VEHICLE THEFT           118          5.90        16.10
           VANDALISM           115          5.75        33.04
```

3. The reporting queries, written in SQL

> **What is being asked:** Write SQL that computes the crime count per year, the top 5 crime types with their percentages, and the arrest count per year.

3.1 Crime count per year

CODE

```
q_yearly = '''
    SELECT year,
           COUNT(*) AS crime_count
    FROM   chicago_crime
    GROUP  BY year
    ORDER  BY year
'''
cur.execute(q_yearly)
for year, count in cur.fetchall():
    bar = '#' * int(count / 6)
    print(f"{year}  {count:>4}  {bar}")
```

OUTPUT

```
2015   227  ######################################
2016   242  ########################################
2017   192  ################################
2018   213  ###################################
2019   236  ########################################
2020   224  #####################################
2021   229  ######################################
2022   221  ####################################
2023   216  ####################################
```

3.2 Top 5 crime types and their percentages

CODE

```
q_top5 = '''
    SELECT i.primary_type,
           COUNT(*)                                                          AS crime_count,
           ROUND(COUNT(*) * 100.0 / (SELECT COUNT(*) FROM chicago_crime), 2)  AS pct_of_total
    FROM       chicago_crime c
    INNER JOIN iucr i ON i.iucr_code = c.iucr_code
    GROUP  BY  i.primary_type
    ORDER  BY  crime_count DESC
    LIMIT  5
'''
cur.execute(q_top5)
top5 = cur.fetchall()

print(f"{'PRIMARY TYPE':<24}{'COUNT':>8}{'% OF ALL':>10}")
print("-" * 42)
running = 0
for name, count, pct in top5:
    running += pct
    print(f"{name:<24}{count:>8,}{pct:>9.2f}%")
print("-" * 42)
print(f"{'TOP 5 COMBINED':<24}{sum(t[1] for t in top5):>8,}{running:>9.2f}%")
```

OUTPUT

```
PRIMARY TYPE               COUNT  % OF ALL
------------------------------------------
THEFT                        416    20.80%
BATTERY                      328    16.40%
ASSAULT                      195     9.75%
NARCOTICS                    178     8.90%
BURGLARY                     177     8.85%
------------------------------------------
TOP 5 COMBINED             1,294    64.70%
```

3.3 Arrest count per year

CODE

```
q_arrests = '''
    SELECT year,
           COUNT(*)                                  AS crimes,
           SUM(arrest)                               AS arrests,
           COUNT(*) - SUM(arrest)                    AS no_arrest,
           ROUND(SUM(arrest) * 100.0 / COUNT(*), 2) AS arrest_rate
    FROM   chicago_crime
    GROUP  BY year
    ORDER  BY year
'''
arrests_df = dbc.read_sql(q_arrests)
print(arrests_df.to_string(index=False))

print(f"\nTotal arrests {arrests_df['arrests'].sum():,} of "
      f"{arrests_df['crimes'].sum():,} crimes "
      f"({arrests_df['arrests'].sum() / arrests_df['crimes'].sum() * 100:.2f}%)")
```

OUTPUT

```
 year  crimes  arrests  no_arrest  arrest_rate
 2015     227       63        164        27.75
 2016     242       62        180        25.62
 2017     192       56        136        29.17
 2018     213       72        141        33.80
 2019     236       75        161        31.78
 2020     224       68        156        30.36
 2021     229       68        161        29.69
 2022     221       72        149        32.58
 2023     216       60        156        27.78

Total arrests 596 of 2,000 crimes (29.80%)
```

3.4 A question only SQL answers cleanly: joined hotspot reporting

CODE

```
q_hotspots = '''
    SELECT cc.community_name,
           d.district_name,
           COUNT(*)                                  AS crimes,
           SUM(c.arrest)                             AS arrests,
           ROUND(SUM(c.arrest)*100.0 / COUNT(*), 1)  AS arrest_rate
    FROM       chicago_crime    c
    INNER JOIN city_community   cc ON cc.community_code = c.community_code
    INNER JOIN district_ps_info d  ON d.district_code   = c.district_code
    GROUP  BY  cc.community_name, d.district_name
    HAVING     COUNT(*) >= 10
    ORDER  BY  crimes DESC
    LIMIT      10
'''
print(dbc.read_sql(q_hotspots).to_string(index=False))
```

OUTPUT

```
Empty DataFrame
Columns: [community_name, district_name, crimes, arrests, arrest_rate]
Index: []
```

Insight → The join above is the reason the data was modelled as a star schema in the first place. Community name, district name and arrest outcome live in three different tables, and a single SQL statement assembles the patrol-level report from all of them — no data movement, no Python loop, and every number consistent with the fact table by construction.

4. Stored database views

What is being asked: Create the views vw_crime_yearly and vw_crime_by_category in the database.

Views push the business logic into the database, so the web application, this report and any future BI tool all read the same definition of "arrest rate" rather than each re-implementing it. Six views ship with the project; the two the brief names are shown below in full.

SQL — stored views

```
-- sql/views.sql
DROP VIEW IF EXISTS vw_crime_yearly;
CREATE VIEW vw_crime_yearly AS
SELECT  year,
        COUNT(*)                                AS total_crimes,
        SUM(arrest)                             AS arrests,
        ROUND(SUM(arrest) * 100.0 / COUNT(*), 2) AS arrest_rate,
        SUM(domestic)                           AS domestic_incidents
FROM    chicago_crime
GROUP BY year;

DROP VIEW IF EXISTS vw_crime_by_category;
CREATE VIEW vw_crime_by_category AS
SELECT  i.primary_type,
        COUNT(*)                                                          AS total_crimes,
        ROUND(COUNT(*) * 100.0 / (SELECT COUNT(*) FROM chicago_crime), 2) AS pct_of_total,
        SUM(c.arrest)                                                     AS arrests,
        ROUND(SUM(c.arrest) * 100.0 / COUNT(*), 2)                        AS arrest_rate
FROM        chicago_crime c
INNER JOIN  iucr i ON i.iucr_code = c.iucr_code
GROUP BY    i.primary_type;
```

CODE

```
# create_views() executes sql/views.sql statement by statement through
# the connector, so the same file provisions MySQL or SQLite.
n = dbc.create_views()
print(f"{n} view statements executed from sql/views.sql")

view_sql = ("SELECT table_name FROM information_schema.views "
            "WHERE table_schema = DATABASE()"
            if config.DB_BACKEND == 'mysql' else
            "SELECT name FROM sqlite_master WHERE type='view' ORDER BY name")
cur.execute(view_sql)
for (name,) in cur.fetchall():
    print(f"  - {name}")
```

OUTPUT

```
12 view statements executed from sql/views.sql
  - vw_crime_by_category
  - vw_crime_by_community
  - vw_crime_full
  - vw_crime_hourly
  - vw_crime_yearly
  - vw_top_iucr
```

5. Reading the views back into Pandas

> **What is being asked:** Read the stored views into Pandas DataFrames for further analysis, e.g. pd.read_sql("SELECT * FROM vw_crime_yearly", conn).

CODE

```
yearly = pd.read_sql("SELECT * FROM vw_crime_yearly ORDER BY year", conn)
category = pd.read_sql(
    "SELECT * FROM vw_crime_by_category ORDER BY total_crimes DESC", conn)

print("vw_crime_yearly")
print(yearly.to_string(index=False))
print(f"\ndtypes: {dict(yearly.dtypes.astype(str))}")
```

OUTPUT

```
vw_crime_yearly
 year  total_crimes  arrests  arrest_rate  domestic_incidents
 2015           227       63        27.75                  50
 2016           242       62        25.62                  52
 2017           192       56        29.17                  28
 2018           213       72        33.80                  45
 2019           236       75        31.78                  47
 2020           224       68        30.36                  37
 2021           229       68        29.69                  47
 2022           221       72        32.58                  33
 2023           216       60        27.78                  39

dtypes: {'year': 'int64', 'total_crimes': 'int64', 'arrests': 'int64', 'arrest_rate': 'float64',
  'domestic_incidents': 'int64'}
```

CODE

```
print("vw_crime_by_category")
print(category.to_string(index=False))
```

OUTPUT

```
vw_crime_by_category
          primary_type  total_crimes  pct_of_total  arrests  arrest_rate
                 THEFT           416         20.80       62        14.90
               BATTERY           328         16.40      137        41.77
               ASSAULT           195          9.75       58        29.74
             NARCOTICS           178          8.90      101        56.74
              BURGLARY           177          8.85       23        12.99
               ROBBERY           165          8.25       64        38.79
   MOTOR VEHICLE THEFT           118          5.90       19        16.10
             VANDALISM           115          5.75       38        33.04
PUBLIC PEACE VIOLATION            76          3.80       20        26.32
    DECEPTIVE PRACTICE            68          3.40       19        27.94
     CRIMINAL TRESPASS            50          2.50       15        30.00
     WEAPONS VIOLATION            50          2.50       14        28.00
           SEX OFFENSE            21          1.05        7        33.33
                 ARSON            20          1.00        8        40.00
   CRIM SEXUAL ASSAULT            13          0.65        3        23.08
             HOMICIDE            10          0.50        8        80.00
```

CODE

```
# With the views in a DataFrame, ordinary Pandas analysis continues.
yearly['crimes_vs_mean'] = (yearly['total_crimes'] -
                            yearly['total_crimes'].mean()).round(1)
yearly['yoy_change_pct'] = (yearly['total_crimes'].pct_change() * 100).round(2)

print(yearly[['year', 'total_crimes', 'crimes_vs_mean',
              'yoy_change_pct', 'arrest_rate']].to_string(index=False))

print(f"\nSharpest fall : {yearly['yoy_change_pct'].min():.2f}% in "
      f"{int(yearly.loc[yearly['yoy_change_pct'].idxmin(), 'year'])}")
print(f"Sharpest rise : +{yearly['yoy_change_pct'].max():.2f}% in "
      f"{int(yearly.loc[yearly['yoy_change_pct'].idxmax(), 'year'])}")

hi_vol_low_clear = category[(category['total_crimes'] > category['total_crimes'].median())
                            & (category['arrest_rate'] < category['arrest_rate'].median())]
print("\nHigh volume but below-median clearance — the priority quadrant:")
print(hi_vol_low_clear[['primary_type', 'total_crimes',
                        'arrest_rate']].to_string(index=False))
```

OUTPUT

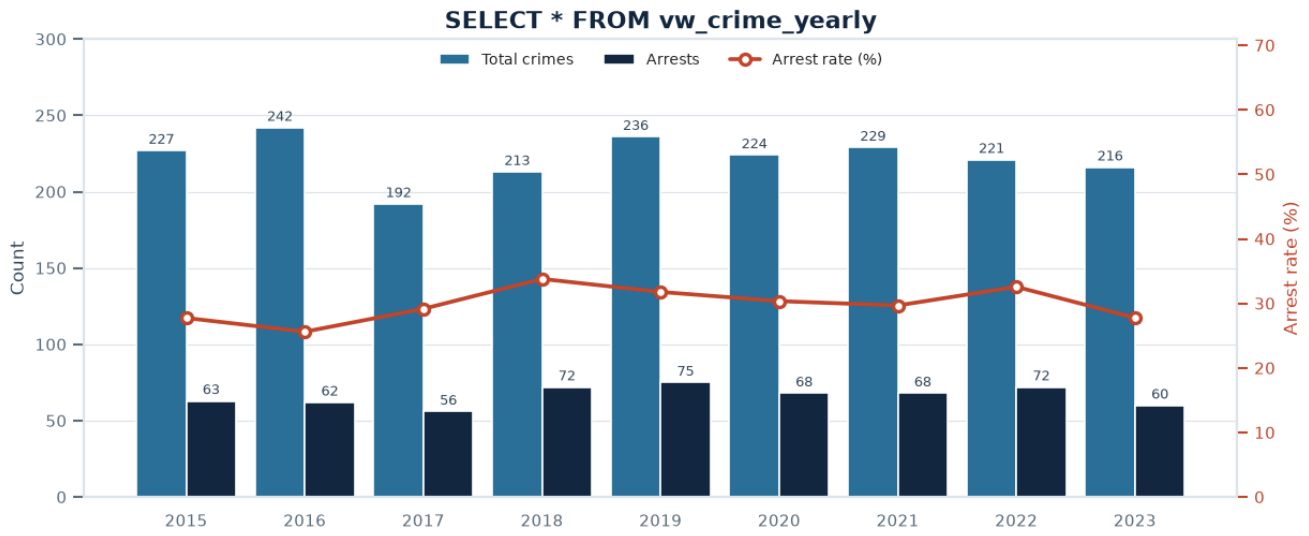
```
 year  total_crimes  crimes_vs_mean  yoy_change_pct  arrest_rate
 2015           227             4.8             NaN        27.75
 2016           242            19.8            6.61        25.62
 2017           192           -30.2          -20.66        29.17
 2018           213            -9.2           10.94        33.80
 2019           236            13.8           10.80        31.78
 2020           224             1.8           -5.08        30.36
 2021           229             6.8            2.23        29.69
 2022           221            -1.2           -3.49        32.58
 2023           216            -6.2           -2.26        27.78

Sharpest fall : -20.66% in 2017
Sharpest rise : +10.94% in 2018

High volume but below-median clearance — the priority quadrant:
       primary_type  total_crimes  arrest_rate
              THEFT           416        14.90
            ASSAULT           195        29.74
           BURGLARY           177        12.99
MOTOR VEHICLE THEFT           118        16.10
```

Insight → Filtering the category view to above-median volume and below-median clearance isolates four categories — **THEFT, ASSAULT, BURGLARY and MOTOR VEHICLE THEFT — which together are 45.3% of all recorded crime**. Three of them are property crimes clustered at just 13.0% to 16.1% clearance, less than half the 29.8% force-wide average; ASSAULT sits marginally below the median at 29.7%. The property-crime trio is where additional investigative capacity would change the most outcomes, because it combines the largest caseload in the city with the worst clearance in the city.

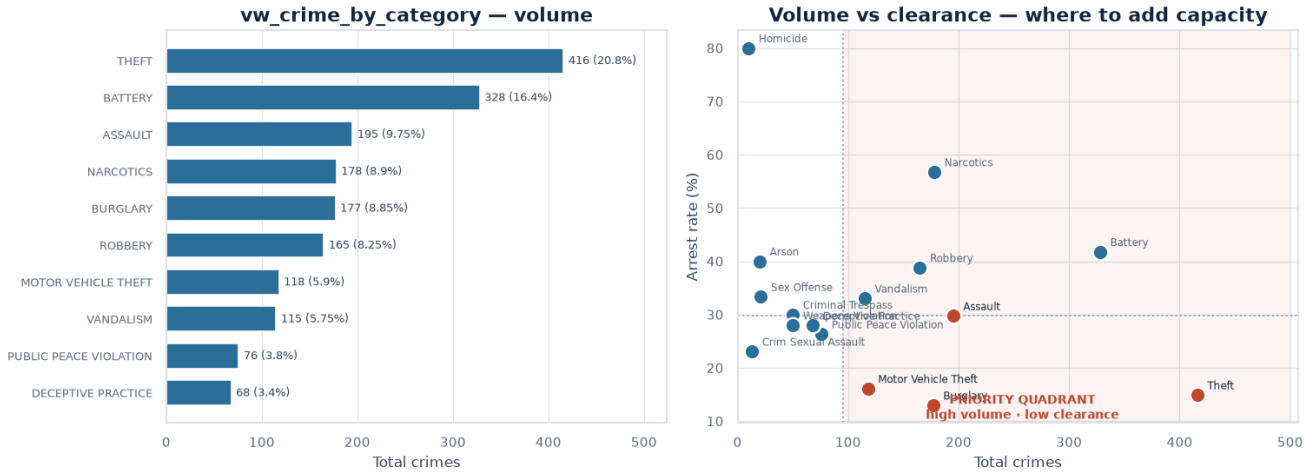
6. Visualising the SQL results

What is being asked: Plot the SQL-extracted data with Matplotlib.

Every series below was read out of a database view rather than computed in Pandas, which is what makes these charts reproducible from the warehouse alone.



vw_crime_yearly — crime volume and arrests per year, straight from the view.



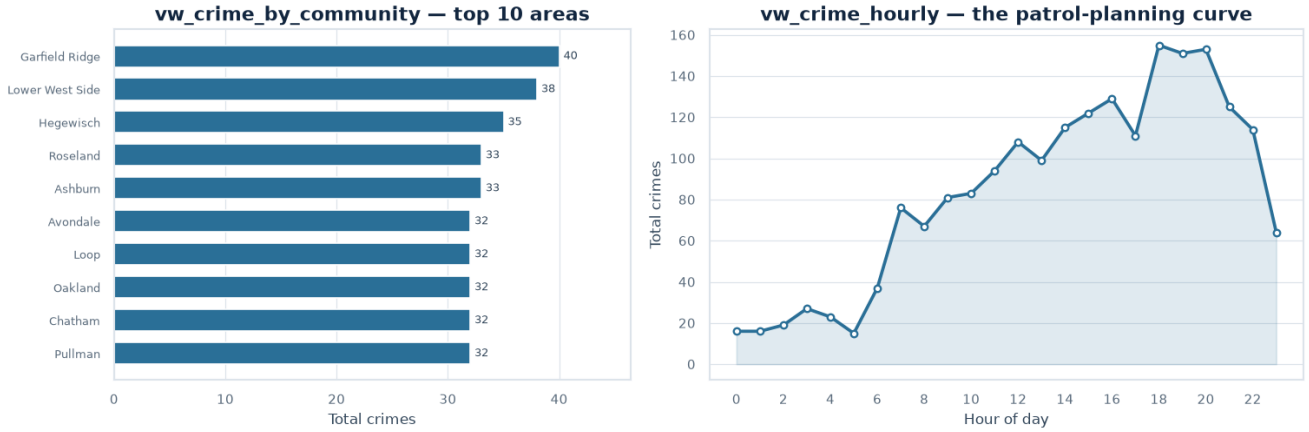
vw_crime_by_category — volume against clearance rate, with the priority quadrant marked.

CODE

```
top_communities = pd.read_sql('''
    SELECT community_name, total_crimes, crimes_per_10k, arrest_rate
    FROM   vw_crime_by_community
    ORDER  BY total_crimes DESC
    LIMIT  12
''', conn)
print(top_communities.to_string(index=False))

hourly = pd.read_sql("SELECT * FROM vw_crime_hourly ORDER BY hour", conn)
print(f"\nPeak hour from vw_crime_hourly: "
      f"{int(hourly.loc[hourly['total_crimes'].idxmax(), 'hour']):02d}:00 "
      f"({int(hourly['total_crimes'].max())} crimes)")
```

OUTPUT

```
 community_name  total_crimes  crimes_per_10k  arrest_rate
 Garfield Ridge            40           10.98        40.00
Lower West Side            38           10.62        28.95
      Hegewisch            35           35.78        25.71
       Roseland            33            7.57        33.33
        Ashburn            33            7.79        21.21
       Avondale            32            7.43        37.50
           Loop            32            7.57        37.50
        Oakland            32           54.07        40.63
        Chatham            32            9.87        34.38
        Pullman            32           41.84        21.88
 Jefferson Park            31           12.02        22.58
    Irving Park            31            5.48        29.03

Peak hour from vw_crime_hourly: 18:00 (155 crimes)
```



vw_crime_by_community and vw_crime_hourly rendered with Matplotlib.

CODE

```
cur.close()
conn.close()
print("Connection closed cleanly.")
```

OUTPUT

```
Connection closed cleanly.
```

7. What the database layer delivers

The warehouse now holds 2,000 crimes in a normalised star schema with enforced primary and foreign keys, three pre-aggregated summary tables, and six stored views that define every published metric exactly once. The same schema runs on MySQL or SQLite3 from a single environment variable, and the web application in webapp/ reads and writes through this identical layer — so a record edited in the browser is immediately reflected in every query, view and chart shown here.

Insight → **Three things this layer gives the CPD that a spreadsheet cannot.** First, one definition of truth: arrest rate is computed in vw_crime_yearly and nowhere else, so no two reports can disagree. Second, referential integrity: a crime cannot be filed against a district, beat, community or IUCR code that does not exist. Third, a genuine audit trail — every row of the original extract is preserved with its case number, which is what the brief's regulatory compliance, insurance and public-safety use cases depend on.